



DAX Fundamentals

Data Analysis Expressions for Power BI & Excel

Chapter 1

Introduction to DAX

Welcome to the world of DAX! This chapter introduces you to one of the most important formula languages used in modern data analysis. By the end of this chapter, you will understand what DAX is, where it is used, how it is structured, and why it is such a powerful tool for working with data.

1.1 What is DAX?

DAX	Data Analysis Expressions. A formula language used in Microsoft Power BI, Excel Power Pivot, and SQL Server Analysis Services (SSAS) to define custom calculations on data stored in tables.
------------	--

DAX allows you to create two types of custom calculations:

- Calculated Columns — new columns added to a table, with a value computed row by row.
- Measures — dynamic calculations that respond to the context of your report (such as filters and slicers).

Real-World Analogy

Think of a data table like a spreadsheet filled with raw transaction records — Order ID, Product, Region, Quantity, Unit Price.

Excel formulas let you compute things cell by cell. But when you have millions of rows and want your calculation to respond automatically to filters — for example, 'show me total revenue only for the North region in Q2' — you need something smarter.

DAX is that smarter tool. You write a formula once, and it recalculates instantly whenever a user applies a filter, selects a region, or changes the time period on a report.

1.2 Where Is DAX Used?

DAX is used in three Microsoft products that share the same underlying data engine called the Tabular Model:

Product	Where DAX Is Used
Power BI Desktop	Building reports and dashboards — the most common platform for DAX today.
Excel Power Pivot	Advanced Excel data modelling with large datasets across multiple tables.
SQL Server Analysis Services (SSAS)	Enterprise-grade data modelling and reporting for large organizations.



Note

This textbook uses Power BI Desktop as the primary platform for examples and screenshots. All DAX formulas work identically in Excel Power Pivot. The interface is slightly different, but the syntax is exactly the same.

1.3 DAX vs Excel Formulas — What Is Different?

If you have used Excel formulas before, you already have a head start. DAX borrows concepts from Excel, but it works differently in a few important ways.

Feature	Excel Formulas	DAX
Operates on	Individual cells (e.g., A1, B3)	Entire columns in data tables
References	Cell references: =A1*B1	Column references: =[Unit Price]*[Quantity]
Responds to filters?	No — the formula is static	Yes — recalculates automatically based on report context
Works across tables?	VLOOKUP needed to link tables	Relationships between tables — no VLOOKUP needed
Best suited for	Small datasets, single tables	Large datasets, multiple related tables, dynamic reports

 Pro Tip

If you already know Excel formulas like SUM, COUNT, AVERAGE, IF — you will recognise many DAX functions. The names and logic are similar. The main difference is that DAX functions work on table columns, not individual cells.

1.4 Understanding the DAX Data Model

Before writing DAX formulas, you need to understand the environment in which DAX works. DAX operates on a Data Model — a collection of tables that are connected to each other through relationships.

1.4.1 Tables and Columns

In Power BI, data is organized into tables. Each table has columns, and each row represents one record.

 Example — Sales Data Model

A typical sales data model might include:

Table: Sales	Columns: Order ID, Date, Product ID, Customer ID, Quantity, Unit Price
Table: Products	Columns: Product ID, Product Name, Category, Cost Price
Table: Customers	Columns: Customer ID, Customer Name, City, Region
Table: Calendar	Columns: Date, Month, Quarter, Year

DAX formulas can reach across these tables using the relationships between them.

1.4.2 Relationships Between Tables

Tables are connected through relationships — similar to how a primary key and foreign key work in a database. For example, the Sales table has a 'Product ID' column that links to the Products table's 'Product ID' column.

This allows a DAX formula in the Sales table to automatically look up the Product Name, Category, or Cost Price from the Products table — without needing any VLOOKUP.

1.5 DAX Syntax Rules

Every DAX formula follows a specific structure. Understanding this structure from the start will prevent confusion and errors.

1.5.1 Basic Syntax Structure

```
// General structure of a DAX measure:  
Measure Name = DAX_Function( arguments )  
  
// General structure of a DAX calculated column:  
Column Name = DAX_Expression_Using_[Column_Name]  
  
// Examples:  
Total Revenue = SUM( Sales[Revenue] )  
Full Name = Customers[First Name] & " " & Customers[Last Name]
```

Figure 1.3 — Basic DAX formula structure

Syntax Element	Example	What It Means
Table name	Sales	The name of the table in your data model
Column reference	Sales[Revenue]	The Revenue column inside the Sales table
Measure name	Total Revenue	The name you assign to your calculation
Function	SUM()	A built-in DAX function that performs a calculation
Text string	" " or "North"	Text values are enclosed in double quotes
Comments	// This is a comment	Double slash marks a comment — ignored by DAX

Common Mistake

A very common beginner error is to reference a column without the table name.

WRONG: = SUM([Revenue]) ← DAX does not know which table's Revenue column

CORRECT: = SUM(Sales[Revenue]) ← Correct: table name included

Always write column references as: TableName[ColumnName]

1.5.2 Where to Write DAX in Power BI

In Power BI Desktop, you write DAX formulas in two places:

For Measures:

- Select a table in the Fields pane.
- Go to the Modeling tab → New Measure. A formula bar appears at the top.
- Type your DAX formula and press Enter.

For Calculated Columns:

- Select a table in the Fields pane.
- Go to the Modeling tab → New Column. A formula bar appears.
- Type your DAX formula and press Enter.



Chapter 2

Calculated Columns vs Measures

DAX provides two types of custom calculations: Calculated Columns and Measures. These are the two building blocks of all DAX work. Understanding the difference between them — what each one does, when to use each one, and how they behave — is the most important foundation in learning DAX.

2.1 Calculated Columns

Calculated Column	A new column added to an existing table in your data model. The formula is evaluated row by row for every record in the table and the result is stored permanently in the model.
--------------------------	--

A calculated column works like adding a new column to your Excel table and writing a formula in it — except you write the formula once and it applies to every row automatically.

Example — Calculated Column

You have a Sales table with two columns: [Unit Price] and [Quantity].
You want to add a new column called [Revenue] that multiplies them.

DAX formula (written in the table): `Revenue = Sales[Unit Price] * Sales[Quantity]`

Power BI evaluates this formula for every row and stores the result in a new Revenue column.
If the Sales table has 50,000 rows, Power BI computes 50,000 individual values.

Key characteristics of Calculated Columns:

- Evaluated row by row — one value is computed per row.
- Results are stored in the data model — they use memory.
- Visible in the Fields pane and can be used in visuals, filters, and slicers.
- Calculated when the data is loaded or refreshed.
- Do not respond to report filters or slicers — the stored value stays fixed.

2.2 Measures

Measure	A dynamic calculation that is evaluated at query time — meaning it calculates its result based on the current context of the report (which filters, slicers, or row/column headers are active).
----------------	---

A measure does not add a column to your table. Instead, it is a reusable formula that calculates a single result, which can change depending on what is selected on the report.

Example — Measure

You create a measure: Total Revenue = SUM(Sales[Revenue])

When you place this measure on a report card — it shows the total revenue for all rows.

When a user selects 'North Region' from a slicer — it instantly recalculates to show only North region revenue.

When placed in a bar chart by Product Category — it shows the revenue for each category separately.

The formula does not change. What changes is the context it is evaluated in.

Key characteristics of Measures:

- Evaluated dynamically — recalculate every time the report context changes.
- Do not store data — they do not consume storage memory.
- Respond to all filters, slicers, and report interactions.
- Best practice: store measures in a dedicated empty table called a 'Measure Table' for easy organization.
- Shown in the Fields pane with a calculator icon.

2.3 Side-by-Side Comparison

Feature	Calculated Column	Measure
What it creates	A new column in a table	A reusable calculation formula
When it calculates	During data load / refresh	Every time a visual is rendered
Evaluated	Row by row (one value per row)	Over the entire filtered dataset
Responds to filters?	No — value is fixed after load	Yes — recalculates with every filter
Stored in model?	Yes — uses memory	No — computed on demand
Visible in table?	Yes — shows as a column	No — shows only in visuals
Use case	Row-level values: categories, flags, bins, labels	Aggregations: totals, averages, percentages, KPIs

2.4 When to Use Each

Use a Calculated Column when...	Use a Measure when...
---------------------------------	-----------------------

You need to classify or label each row (e.g., 'High Value' vs 'Low Value')	You want to show totals, counts, or averages on a visual
You need to combine text from two columns (e.g., Full Name)	You want the result to change based on a slicer or filter
You need a value that will be used in a slicer or as a filter	You are building KPIs (Key Performance Indicators)
You need to categorize rows into buckets (e.g., age groups)	You are creating percentage or ratio calculations

**Pro Tip — Best Practice**

In professional Power BI reports, Measures are used far more often than Calculated Columns. As a general rule: if the calculation produces a summary (total, average, count, ratio) — it should be a measure. If it produces a row-level value (a label, a bin, a derived column) — it should be a calculated column.



Chapter 3

Basic DAX Functions — SUM, COUNT, AVERAGE

DAX has hundreds of functions, but most real-world reports are built using a small core set. This chapter covers the three most fundamental aggregation functions — SUM, COUNT, and AVERAGE — along with their variants. These functions are the starting point for every DAX developer.

3.1 The SUM Function

SUM(ColumnName)

Adds up all the values in a numeric column. Returns the total of all non-blank values.

Syntax:

```
Measure Name = SUM( TableName[ColumnName] )
```

Example — Total Revenue

Your Sales table has a [Revenue] column with individual transaction amounts. You want a measure that shows the total revenue across all transactions.

```
Total Revenue = SUM( Sales[Revenue] )
```

When placed on a report card → shows overall total.

When placed in a chart by Region → shows revenue per region.

When a 'Product Category' filter is applied → shows revenue for that category only.

```
// Common SUM measures:
Total Revenue = SUM( Sales[Revenue] )
Total Units Sold = SUM( Sales[Quantity] )
Total Cost = SUM( Sales[Cost] )
Total Discount = SUM( Sales[Discount Amount] )
```

Figure 3.1 — Common SUM measures used in sales reports

SUMX — The Row-by-Row Version:

Sometimes you need to multiply two columns first and then sum the result. SUM cannot do this — you use SUMX instead.

```
// SUMX evaluates an expression row by row, then sums the results:
Total Revenue = SUMX( Sales, Sales[Unit Price] * Sales[Quantity] )

// This is equivalent to creating a calculated column Revenue = [Unit Price]*[Quantity]
// and then doing SUM(Sales[Revenue]) — but SUMX does it without storing an extra column.
```

Figure 3.3 — SUMX evaluates row-by-row and then aggregates

3.2 The COUNT Function Family

DAX has several COUNT variants, each designed for a specific counting scenario. It is important to know which one to use.

Function	What It Counts	Typical Use Case
COUNT	Numeric values only (ignores blanks and text)	Count number of transactions with a recorded amount
COUNTA	All non-blank values (numbers, text, dates)	Count how many rows have any value in a column
COUNTBLANK	Blank (empty) cells only	Count how many rows are missing a value
COUNTROWS	All rows in a table (no column needed)	Count total number of transactions or records
DISTINCTCOUNT	Unique values only (no duplicates)	Count how many unique customers, products, or regions

```
// COUNT — counts only rows where Sales[Revenue] has a number:
Transactions with Revenue = COUNT( Sales[Revenue] )
```

```
// COUNTA — counts all non-blank order IDs:
Total Orders = COUNTA( Sales[Order ID] )
```

```
// COUNTROWS — counts every row in the Sales table:
Total Transactions = COUNTROWS( Sales )
```

```
// DISTINCTCOUNT — counts how many unique customers placed orders:
Unique Customers = DISTINCTCOUNT( Sales[Customer ID] )
```

```
// COUNTBLANK — counts how many rows have no shipping date:
Orders Not Shipped = COUNTBLANK( Sales[Shipping Date] )
```

Figure 3.4 — Different COUNT function variants with examples



Example — Choosing the Right COUNT

Scenario: You want to know how many unique products were sold in the last quarter.

COUNT(Sales[Product ID]) → Counts total rows where Product ID is a number (not useful here)

COUNTA(Sales[Product ID]) → Counts all non-blank Product IDs (includes duplicates)

DISTINCTCOUNT(Sales[Product ID]) → Counts unique products only ← CORRECT

The answer: use DISTINCTCOUNT when you want unique counts.

3.3 The AVERAGE Function

AVERAGE(ColumnName)

Calculates the arithmetic mean of all numeric values in a column. Blank values are automatically excluded from both the sum and the count.

```
// AVERAGE — mean of all values in a column:
Avg Revenue per Order = AVERAGE( Sales[Revenue] )
Avg Unit Price       = AVERAGE( Sales[Unit Price] )
Avg Delivery Days    = AVERAGE( Orders[Delivery Days] )
```

Figure 3.5 — Common AVERAGE measure examples



Example — Average Order Value

You want to show the average value of a single order to track customer spend.

Avg Order Value = AVERAGE(Sales[Revenue])

On a monthly trend chart, this measure shows the average order value for each month.

If filtered by 'Premium' customer segment, it shows the average order value for premium customers only.

AVERAGEX — The Row-by-Row Version:

Just like SUMX, AVERAGEX lets you compute an expression row by row first, then calculate the average of those results.

```
// AVERAGEX: calculate margin per row, then average those margins:
Avg Margin % = AVERAGEX(
    Sales,
    DIVIDE( Sales[Revenue] - Sales[Cost], Sales[Revenue] )
)
```

```
// This gives the average margin percentage calculated per order.
// Note: DIVIDE is used instead of / to safely handle division by zero.
```

Figure 3.6 — AVERAGEX computing a row-level expression before averaging

3.4 Other Essential Basic Functions

3.4.1 MIN and MAX

```
// MIN — returns the smallest value in a column:
```

```
Lowest Unit Price = MIN( Sales[Unit Price] )  
Earliest Order Date = MIN( Sales[Order Date] )  
  
// MAX — returns the largest value in a column:  
Highest Revenue Order = MAX( Sales[Revenue] )  
Latest Order Date = MAX( Sales[Order Date] )
```

Figure 3.7 — MIN and MAX function examples

3.4.2 DIVIDE — Safe Division

In DAX, dividing by zero causes an error. The DIVIDE function handles this safely by returning a blank or a specified alternate result when the denominator is zero.

```
// Standard division (will error if denominator is 0):  
Profit Margin = Sales[Profit] / Sales[Revenue] // RISKY  
  
// DIVIDE handles zero division safely:  
Profit Margin = DIVIDE( Sales[Profit], Sales[Revenue] ) // Returns blank if Revenue is 0  
Profit Margin = DIVIDE( Sales[Profit], Sales[Revenue], 0 ) // Returns 0 if Revenue is 0
```

Figure 3.8 — Using DIVIDE for safe division



Pro Tip — Always Use DIVIDE

In professional DAX, always use DIVIDE(numerator, denominator) instead of the / operator for percentage and ratio calculations. It prevents your report from showing errors when the denominator is zero.

Chapter 4

The CALCULATE Function

CALCULATE is the single most important and most powerful function in all of DAX. Once you understand CALCULATE, you unlock the ability to build almost any business calculation — from year-to-date totals to same-period comparisons to custom segment analysis.

4.1 What is CALCULATE?

CALCULATE

Evaluates an expression (usually a measure) in a modified filter context. It allows you to override, add to, or remove the filters that are currently active on a report.

The key word here is context. In Power BI, every visual exists inside a filter context — the combination of all active filters, slicers, and row/column headers that determine which rows of data are being considered.

CALCULATE lets you step outside the current context and apply your own custom filter when evaluating a calculation.

4.2 Syntax of CALCULATE

```
Measure Name = CALCULATE(  
    <Expression>,      // The measure or calculation to evaluate  
    <Filter 1>,         // Optional: a filter to apply  
    <Filter 2>,         // Optional: additional filters (AND logic between filters)  
    ...  
)
```

Figure 4.1 — CALCULATE syntax structure

The Expression is the calculation you want to perform — usually a measure like SUM, COUNT, or AVERAGE.

The Filters are conditions that modify which rows are included in the calculation.

4.3 CALCULATE — Simple Examples

4.3.1 Filtering by a Single Value



Example — Revenue for North Region Only

You want a measure that always shows the North region revenue, regardless of what filter is active on the report.

```
North Revenue = CALCULATE( SUM(Sales[Revenue]), Sales[Region] = "North" )
```

Even if a user selects 'South' from a region slicer, this measure will continue to show North revenue — because CALCULATE overrides the Region filter with its own.

```
// Revenue for a specific region:
North Revenue = CALCULATE( SUM(Sales[Revenue]), Sales[Region] = "North" )

// Revenue for a specific product category:
Electronics Revenue = CALCULATE( SUM(Sales[Revenue]), Products[Category] = "Electronics" )

// Count of orders with status 'Delivered':
Delivered Orders = CALCULATE( COUNTROWS(Sales), Sales[Status] = "Delivered" )
```

Figure 4.2 — CALCULATE with single-value filters

4.3.2 Filtering by Multiple Values

```
// Revenue for either Electronics OR Furniture (using FILTER — covered in Chapter 5):
Electronics or Furniture Revenue = CALCULATE(
    SUM(Sales[Revenue]),
    Products[Category] IN { "Electronics", "Furniture" }
)

// Revenue for North region AND Q1 only:
North Q1 Revenue = CALCULATE(
    SUM(Sales[Revenue]),
    Sales[Region] = "North",
    Sales[Quarter] = "Q1"
)
// Multiple filter arguments use AND logic — both conditions must be true.
```

Figure 4.3 — CALCULATE with multiple filter conditions

4.4 CALCULATE with ALL — Removing Filters

One of the most powerful uses of CALCULATE is to remove filters that are currently active. You do this using the ALL function inside CALCULATE.

ALL()

Removes all filters from a table or specific column, returning all rows as if no filter exists. Commonly used inside CALCULATE to compute totals or percentages.



Example — Percentage of Total Revenue

You want each product category to show its percentage share of the overall total revenue.

Total Revenue = SUM(Sales[Revenue]) // This responds to category filter

```
% of Total Revenue = DIVIDE(
    SUM( Sales[Revenue] ),
    CALCULATE( SUM(Sales[Revenue]), ALL(Sales) ) // ALL removes all filters on Sales
)
```

The denominator always returns the grand total, regardless of what category is selected.

The numerator returns only the current category's revenue.

Result: each row shows its percentage share of the total.

```
// % of Total Revenue by Category:
% of Total = DIVIDE(
    SUM( Sales[Revenue] ),
    CALCULATE( SUM(Sales[Revenue]), ALL(Sales) )
)

// % of Total — remove filter from only one column:
% of Region Total = DIVIDE(
    SUM( Sales[Revenue] ),
    CALCULATE( SUM(Sales[Revenue]), ALL(Sales[Product]) )
)
// ALL(Sales[Product]) removes only the Product filter — Region filter stays active.
```

Figure 4.4 — Using ALL inside CALCULATE to compute percentage of total

4.5 Understanding Filter Context — The Key Concept

To truly master CALCULATE, you need to understand what filter context means.



Understanding Filter Context

Imagine your report has a bar chart showing Revenue by Region. Power BI evaluates your 'Total Revenue' measure once for each region — each evaluation happens in a filter context where only that region's rows are visible.

North context: → Total Revenue evaluates SUM(Sales[Revenue]) where Region = North

South context: → Total Revenue evaluates SUM(Sales[Revenue]) where Region = South

East context: → Total Revenue evaluates SUM(Sales[Revenue]) where Region = East

CALCULATE allows you to modify this automatically-applied context and add your own rules.



Note

CALCULATE can also use pre-built time intelligence functions as its filter arguments — such as SAMEPERIODLASTYEAR, DATEYTD, DATESMTD. These are covered in advanced DAX topics.



Chapter 5

The FILTER Function

The FILTER function is one of the most versatile tools in DAX. It is commonly used inside CALCULATE to apply complex, multi-condition filters that go beyond what simple column filter arguments can do.

5.1 What is FILTER?

FILTER(Table, Condition)

Returns a filtered version of a table, containing only the rows where the specified condition is TRUE. FILTER is a table function — it returns a table, not a single value. It is most commonly used as an argument inside CALCULATE.

```
// FILTER syntax:
FILTER( <Table>, <Condition> )

// Example: FILTER returns only the rows where Region is North:
FILTER( Sales, Sales[Region] = "North" )

// This returns a table of rows — not a number.
// To get a number, wrap it in CALCULATE or another aggregation.
```

Figure 5.1 — Basic FILTER syntax

5.2 Using FILTER Inside CALCULATE

FILTER is most powerful when used as the filter argument inside CALCULATE. This allows you to apply row-level conditions that would be impossible with simple equality filters.

5.2.1 Filter by a Threshold Value

Example — Revenue from High-Value Orders Only

You want to measure total revenue from orders where the individual order amount exceeded 10,000.

```
Revenue from Large Orders = CALCULATE(
    SUM( Sales[Revenue] ),
    FILTER( Sales, Sales[Revenue] > 10000 )
)
```

Simple filter arguments like `Sales[Revenue] > 10000` cannot be used directly in CALCULATE. FILTER solves this by creating a filtered table on the fly.

```
// Revenue only from orders above 10,000:
Large Order Revenue = CALCULATE(
    SUM( Sales[Revenue] ),
    FILTER( Sales, Sales[Revenue] > 10000 )
)

// Count of premium customers (those who spent more than 50,000 total):
Premium Customer Count = CALCULATE(
    DISTINCTCOUNT( Sales[Customer ID] ),
    FILTER( Sales, Sales[Revenue] > 50000 )
)

// Revenue from orders placed in 2024:
Revenue 2024 = CALCULATE(
    SUM( Sales[Revenue] ),
    FILTER( Sales, YEAR( Sales[Order Date] ) = 2024 )
)
```

Figure 5.2 — FILTER used with threshold and date conditions inside CALCULATE

5.2.2 FILTER with Multiple Conditions

You can combine multiple conditions inside FILTER using && (AND) and || (OR) operators.

```
// && means AND — both conditions must be true:
North Large Orders = CALCULATE(
    SUM( Sales[Revenue] ),
    FILTER(
        Sales,
        Sales[Region] = "North" && Sales[Revenue] > 10000
    )
)

// || means OR — at least one condition must be true:
North or South Revenue = CALCULATE(
    SUM( Sales[Revenue] ),
    FILTER(
        Sales,
        Sales[Region] = "North" || Sales[Region] = "South"
    )
)
```

Figure 5.3 — FILTER with AND (&&) and OR (||) conditions

5.3 FILTER vs Direct Filter Arguments in CALCULATE

Power BI beginners often wonder: when should I use a direct filter argument, and when do I need FILTER?

Scenario	Direct Filter Argument	Use FILTER Instead
----------	------------------------	--------------------

Exact match on a column value	<code>CALCULATE(SUM(Sales[Rev]), Sales[Region] = "North")</code>	Not needed — direct filter works fine
Comparison (>, <, >=)	Cannot be done directly	<code>FILTER(Sales, Sales[Rev] > 10000)</code>
Multiple conditions with AND/OR	Only AND between separate filter args	<code>FILTER(Sales, condition1 && condition2)</code>
Condition involving a calculated expression	Cannot be done directly	<code>FILTER(Sales, YEAR(Sales[Date]) = 2024)</code>

 Pro Tip — Performance Consideration

FILTER scans every row of the table it is applied to. For very large tables (millions of rows), this can slow down your report.

When possible, prefer simple direct filter arguments in CALCULATE for better performance.

Use FILTER only when the condition requires a comparison, an expression, or multiple OR conditions.

5.4 FILTER with Related Tables

FILTER becomes even more powerful when used with the RELATED function to reach values in a related table.

```
// Revenue from Electronics category (category is in the Products table):
```

```
Electronics Revenue = CALCULATE(
    SUM( Sales[Revenue] ),
    FILTER(
        Sales,
        RELATED( Products[Category] ) = "Electronics"
    )
)
```

```
// RELATED() follows the relationship from Sales to Products
```

```
// and returns the Category value for each row in Sales.
```

Figure 5.4 — Using FILTER with RELATED to access related table columns

Chapter 6

Aggregations & Simple KPIs

Now that you understand the core DAX functions and the CALCULATE-FILTER pattern, this chapter shows you how to combine these concepts to build meaningful business metrics — known as KPIs (Key Performance Indicators). These are the measurements that appear on executive dashboards and management reports.

6.1 What Is a KPI?

KPI (Key Performance Indicator)	A measurable value that demonstrates how effectively an objective is being achieved. In Power BI, KPIs are DAX measures that summarize business performance — such as total revenue, profit margin, customer count, or growth rate.
--	---

 **Example KPIs in Business**

Sales KPIs: Total Revenue, Units Sold, Avg Order Value, Revenue Growth %
Finance KPIs: Gross Profit, Profit Margin %, Operating Cost
Customer KPIs: Unique Customers, Customer Retention Rate, Avg Revenue per Customer
Operations KPIs: On-Time Delivery %, Avg Delivery Days, Return Rate

6.2 Building a Core KPI Measure Set

The following is a professional set of DAX measures for a sales dataset. These measures form the foundation of most real-world Power BI dashboards.

6.2.1 Revenue Measures

```
// 1. Total Revenue — sum of all revenue:  
Total Revenue = SUM( Sales[Revenue] )  
  
// 2. Total Cost — sum of all costs:  
Total Cost = SUM( Sales[Cost] )  
  
// 3. Gross Profit — revenue minus cost:  
Gross Profit = [Total Revenue] - [Total Cost]  
// Note: measures can reference other measures using just [MeasureName]  
  
// 4. Profit Margin % — profit as a percentage of revenue:  
Profit Margin % = DIVIDE( [Gross Profit], [Total Revenue] )
```

Figure 6.1 — Core revenue and profitability measures

6.2.2 Volume and Customer Measures

```
// 5. Total Transactions — count of all orders:  
Total Transactions = COUNTROWS( Sales )  
  
// 6. Total Units Sold — sum of quantity:  
Total Units Sold = SUM( Sales[Quantity] )  
  
// 7. Unique Customers — distinct customer count:  
Unique Customers = DISTINCTCOUNT( Sales[Customer ID] )  
  
// 8. Avg Order Value — mean revenue per transaction:  
Avg Order Value = DIVIDE( [Total Revenue], [Total Transactions] )  
  
// 9. Avg Revenue per Customer:  
Avg Revenue per Customer = DIVIDE( [Total Revenue], [Unique Customers] )
```

Figure 6.3 — Volume, count, and customer-level KPI measures

6.2.3 Conditional KPIs Using CALCULATE

```
// 10. Revenue from Top-Tier Products (price > 500):  
Premium Product Revenue = CALCULATE(  
    SUM( Sales[Revenue] ),  
    FILTER( Sales, Sales[Unit Price] > 500 )  
)  
  
// 11. % Revenue from Premium Products:  
Premium Revenue % = DIVIDE( [Premium Product Revenue], [Total Revenue] )  
  
// 12. Delivered Order Count:  
Delivered Orders = CALCULATE(  
    COUNTROWS( Sales ),  
    Sales[Status] = "Delivered"  
)  
  
// 13. On-Time Delivery % (orders delivered within 5 days):  
On-Time Delivery % = DIVIDE(  
    CALCULATE( COUNTROWS(Sales), FILTER(Sales, Sales[Delivery Days] <= 5) ),  
    [Total Transactions]  
)
```

Figure 6.4 — Conditional KPIs using CALCULATE and FILTER

6.3 Formatting Measures in Power BI

After creating a measure, you should format it appropriately so it displays correctly in visuals. For example, a revenue measure should show currency formatting, and a percentage measure should show with a % symbol.

How to format a measure:

- Click on the measure name in the Fields pane to select it.
- In the Measure Tools tab (appears when a measure is selected), look for the Format section.
- Choose Currency, Percentage, Whole Number, Decimal, etc.
- Set the number of decimal places as needed.

Measure Type	Recommended Format	Example Display
Revenue, Cost, Profit	Currency	₹ 12,45,000 or \$ 1,245,000
Profit Margin %, Growth %	Percentage (2 decimal places)	23.45%
Count of orders/customers	Whole Number	4,521
Average Order Value	Currency (2 decimal places)	₹ 3,450.75

6.4 Organising Measures — The Measure Table

As your report grows, you will create dozens of measures. Keeping them organized is essential for maintaining and sharing reports professionally.

Best practice — create a dedicated Measure Table:

- In Power BI Desktop, go to the Home tab → Enter Data.
- Create an empty table with a single column. Name the table '_Measures' (the underscore pushes it to the top of the Fields pane).
- Click Load.
- Move all your measures into this table by right-clicking each measure → Move to table → _Measures.



Pro Tip — Naming Conventions

Professional Power BI developers follow consistent naming conventions for measures:

Revenue measures: prefix with amounts or totals → 'Total Revenue', 'Gross Profit'
 Ratio/percentage: suffix with % → 'Profit Margin %', 'Growth %'
 Counts: prefix with # or Total → '# Customers', 'Total Orders'
 Averages: prefix with Avg → 'Avg Order Value'

Consistent naming makes it easy for anyone who opens the file to understand each measure.

Chapter 7

Practice Exercise

This chapter provides a comprehensive, step-by-step practice exercise that applies all the concepts covered in this module. You will work with a realistic sales dataset and build a complete set of DAX measures — progressing from basic aggregations to CALCULATE-based KPIs.

7.1 Dataset Description

For this exercise, you will use a simulated retail sales dataset loaded into Power BI Desktop. The data model contains four related tables:

Table	Key Columns	Description
Sales	Order ID, Date, Product ID, Customer ID, Quantity, Unit Price, Revenue, Cost, Status, Delivery Days	One row per transaction
Products	Product ID, Product Name, Category, Sub-Category	One row per product
Customers	Customer ID, Customer Name, City, Region, Segment	One row per customer
Calendar	Date, Day, Month, Month Name, Quarter, Year	One row per calendar date

Relationships:

- Sales[Product ID] → Products[Product ID] (Many-to-One)
- Sales[Customer ID] → Customers[Customer ID] (Many-to-One)
- Sales[Date] → Calendar[Date] (Many-to-One)

7.2 Part A — Calculated Columns

Before creating measures, add the following calculated columns to enrich the data model.

Step A1 — Add a Revenue Column to the Sales Table

The dataset has Quantity and Unit Price but no Revenue column. Create it.

```
// In the Sales table, create a new calculated column:
```

```
Revenue = Sales[Unit Price] * Sales[Quantity]
```

- Go to the Data view in Power BI.
- Select the Sales table from the Fields pane.
- Modeling tab → New Column. Type the formula above. Press Enter.

Step A2 — Add a Profit Column

```
Profit = Sales[Revenue] - Sales[Cost]
```

Step A3 — Add a Customer Segment Label Column to Customers Table

Classify customers based on their total spend. For this exercise, use a simple rule: create a 'Tier' column.

```
// In the Customers table — classifying by region group as a label example:
Region Group = IF(
    Customers[Region] = "North" || Customers[Region] = "East",
    "Zone A",
    "Zone B"
)
```

Figure 7.2 — Calculated columns using IF logic

7.3 Part B — Basic Aggregation Measures

Create a '_Measures' table first (Enter Data → empty table named _Measures → Load). Then create all measures inside it.

```
// M01 — Total Revenue:
Total Revenue = SUM( Sales[Revenue] )

// M02 — Total Cost:
Total Cost = SUM( Sales[Cost] )

// M03 — Total Profit:
Total Profit = SUM( Sales[Profit] )

// M04 — Total Transactions:
Total Transactions = COUNTROWS( Sales )

// M05 — Total Units Sold:
Total Units Sold = SUM( Sales[Quantity] )

// M06 — Unique Customers:
Unique Customers = DISTINCTCOUNT( Sales[Customer ID] )

// M07 — Avg Order Value:
Avg Order Value = DIVIDE( [Total Revenue], [Total Transactions] )
```



```
// M08 — Profit Margin %:  
Profit Margin % = DIVIDE( [Total Profit], [Total Revenue] )
```

Figure 7.3 — Core aggregation measures (M01 to M08)

7.4 Part C — CALCULATE-Based Measures

```
// M09 — Revenue for Electronics Category:  
Electronics Revenue = CALCULATE(  
    [Total Revenue],  
    Products[Category] = "Electronics"  
)  
  
// M10 — Electronics % of Total Revenue:  
Electronics Revenue % = DIVIDE( [Electronics Revenue], [Total Revenue] )  
  
// M11 — Revenue from Premium Segment Customers:  
Premium Segment Revenue = CALCULATE(  
    [Total Revenue],  
    Customers[Segment] = "Premium"  
)  
  
// M12 — Revenue from Orders > 5,000 (large orders):  
Large Order Revenue = CALCULATE(  
    [Total Revenue],  
    FILTER( Sales, Sales[Revenue] > 5000 )  
)
```

Figure 7.5 — CALCULATE-based measures for segment and conditional analysis

7.5 Part D — KPI Measures

```
// M13 — Delivered Order Count:  
Delivered Orders = CALCULATE(  
    [Total Transactions],  
    Sales[Status] = "Delivered"  
)  
  
// M14 — Delivery Rate %:  
Delivery Rate % = DIVIDE( [Delivered Orders], [Total Transactions] )  
  
// M15 — On-Time Delivery % (within 5 days):  
On-Time Delivery % = DIVIDE(  
    CALCULATE( COUNTROWS(Sales), FILTER(Sales, Sales[Delivery Days] <= 5) ),  
    [Total Transactions]  
)  
  
// M16 — Avg Revenue per Customer:  
Avg Revenue per Customer = DIVIDE( [Total Revenue], [Unique Customers] )
```

Figure 7.6 — Operational KPI measures

7.6 Part E — Build the Dashboard

Once all measures are created, assemble a simple one-page dashboard in Power BI:

1. Add 4 Card visuals in a row at the top: Total Revenue, Total Profit, Profit Margin %, Unique Customers.
2. Add a Bar Chart: Axis = Products[Category], Values = Total Revenue. This shows revenue by category.
3. Add a Clustered Column Chart: Axis = Calendar[Month Name], Values = Total Revenue. This shows monthly trend.
4. Add a Table visual: Rows = Customers[Region], Columns = Total Revenue, Total Profit, Profit Margin %, Unique Customers.
5. Add a Slicer: Field = Calendar[Year]. Test that all measures respond correctly when a year is selected.
6. Add a Slicer: Field = Products[Category]. Verify that the Electronics Revenue measure does NOT change when you filter by other categories (because it is hardcoded with CALCULATE).

Expected Observations from the Dashboard

When you select '2024' from the Year slicer:

- Total Revenue, Total Profit, and Unique Customers all update to show 2024 values only.
- Electronics Revenue also updates (the CALCULATE filter adds to the Year filter).

When you select 'Furniture' from the Category slicer:

- Total Revenue updates to show only Furniture revenue.
- Electronics Revenue stays the same — it ignores the Category slicer because CALCULATE overrides it.

This demonstrates the core concept of filter context and how CALCULATE interacts with it.

Module Summary

In this module, you have built a complete foundation in DAX — from understanding the data model, to writing basic aggregations, to constructing context-aware business KPIs. Here is a recap of the key concepts covered:

- DAX (Data Analysis Expressions) is the formula language used in Power BI and Excel Power Pivot to create custom calculations on tabular data.
- DAX operates on columns within tables — not individual cells — and is context-aware, meaning calculations respond automatically to filters and slicers.
- Calculated Columns add a new, stored value to each row of a table. They are computed row by row and do not respond to report filters.
- Measures are dynamic formulas that recalculate based on the current filter context of the report. They are used for aggregations, KPIs, and ratios.
- The basic aggregation functions — SUM, COUNT, COUNTA, COUNTROWS, DISTINCTCOUNT, AVERAGE, MIN, MAX — form the building blocks of all DAX reports.
- CALCULATE is the most powerful DAX function. It evaluates an expression inside a modified filter context, allowing you to override, add, or remove filters.
- FILTER returns a filtered version of a table and is used inside CALCULATE for complex, row-level conditions involving comparisons, expressions, and multiple OR conditions.
- KPIs are meaningful business metrics built from combinations of DAX measures. Well-named, well-formatted, and well-organized measures are the mark of professional Power BI reports.

© DAX Fundamentals Module | Confidential Course Material